

Creating A Simple Web Service

Let's get our hands dirty by creating a simple web app which converts an absolute date into a relative date. So given a date like 11-02-2017 it will say something like "in 2 years".

To build this app we will use two very popular packages. The first is HAPI, which is "A rich framework for building applications and services". The other is library called "moment" that lets you work with dates and times, and includes the ability to compare date.

While HAPI is a pure Node.js package that is designed to run on the server side, moment is pretty much a JavaScript library that can run in the browser or on Node.js.

You should start by creating a directory for this project, and running `npm init` in it. Just stick with the defaults by pressing Enter at all the prompts.

With this done you should install the packages we mentioned by running the following two commands:

```
$ npm install --save hapi
```

```
$ npm install --save moment
```

Finally create a file called `index.js` in which you can place the following code:

```
'use strict';
const Hapi = require('hapi');
const moment = require('moment');
const server = new Hapi.Server();
server.connection({ port: 3000 });
server.route({
  method: 'GET',
  path: '/when/{date}',
  handler: function (request, reply) {
    reply(moment(request.params.date, "DD-MM-YYYY").
      fromNow());
  }
});
server.start();
```

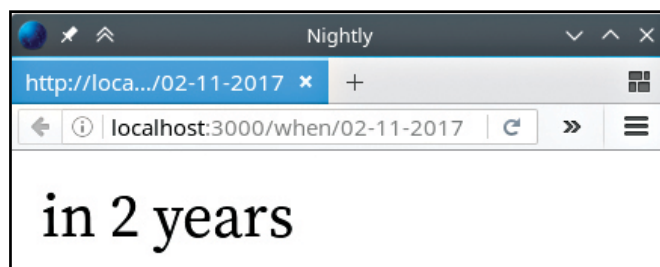
Explaining the intricacies of creating Hapi apps is out of the scope of this article, but here's the gist of what's happening here:

- We first import both the libraries we need.
- We create a new Hapi server that will run out app.
- We specify that this server will use port 3000

- We set up a route on this server which can be accessed at '/when/' followed by the date
- We have some code that is run whenever someone visits such a URL
- We start the server

The function that is run when someone visits our app with a date simply uses the moment library to convert a data string in the format 'DD-MM-YYYY' into a date object, and then we use moment's `fromNow` method to convert this into a relative date. This is sent back as a reply to the browser.

If you now run this `index.js` using `node`, you will be able to open this app in your browser at `http://localhost:3000/` or `http://127.0.0.1:3000/`. That particular page has nothing to show and will probably show a 404 error, however if you visit a URL like `http://localhost:3000/when/02-11-2017` you should see the browser display the simple text 'in 2 years'. Unless of course you are reading this in the future and it shows 'in 1 year' or '2 years ago'. Funny how relative, relative dates can be.



Our HAPI-based web app running in Firefox. We've zoomed it in here to make it more visible.

Conclusion

There is a lot to like about Node.js other than simply reusing JavaScript for server-side development. Many of the architectural decisions made by Node.js make it especially good for handling a large number of connections and in handling real-time web applications.

Even if you only want to develop browser-based client-side applications and don't want to use JavaScript on the server side, knowing a bit about Node.js and npm can be a huge help. There are tonnes of tools like gulp, grunt and webpack that make it easy to set up a pipeline for handling complex web applications. >

*Nodes of Interest



*JXCore

>>JXCore is an open source alternative to Node.js that is compatible with Node.js but can use Mozilla's SpiderMonkey JavaScript engine, and works on more platforms.

<http://dgit.in/JxCoreOS>



*Webpack

>>If you want to do front-end development, webpack is great for bringing all your code and assets together and packaging it for deployment.

<http://dgit.in/WbPck>



*NVM

>>NVM or Node Version Manager is a great tool for OSX and Linux users to manage multiple Node.js installs and switch between them as and when required.

<http://dgit.in/NdVMan>